

Курс: Углубленное машинное обучение и искусственный интеллект в Python

Практическое задание: Автокодировщики (Autoencoders) и снижение размерности

Цель работы: Понять принцип работы и архитектуру автокодировщиков (Autoencoders), научиться реализовывать симметричные сети типа «песочные часы» (Bottleneck) на полносвязных слоях в TensorFlow/Keras, а также работать с табличными или векторизованными данными из Hugging Face Hub.

Шаг 1. Выбор датасета на Hugging Face

Вам необходимо подобрать числовой или векторизованный датасет. Поскольку мы строим полносвязный автокодировщик, входные данные должны представлять собой плоский вектор (одномерный массив чисел).

Доступные варианты на выбор:

- **Вариант А (Табличные данные):** Выберите датасет с непрерывными числовыми признаками. Отличным примером послужат классические датасеты wine, iris (из раздела табличных данных на HF) или любой аналогичный датасет с физическими/финансовыми параметрами.
- **Вариант В (Векторизованные данные):** Выберите датасет, в котором текстовые данные или изображения уже предобработаны и представлены в виде готовых эмбеддингов (векторных представлений предложений).

Шаг 2. Проектирование архитектуры автокодировщика

Вам необходимо собрать нейронную сеть, которая будет постепенно уменьшать размерность входного вектора до латентного пространства («узкое горлышко»), а затем симметрично восстанавливать его до исходной размерности.

1. Encoder (Кодировщик):

- Входной слой: `layers.Input(shape=(num_features,))`, где `num_features` —

количество исходных признаков.

- Скрытые слои: 1–2 полносвязных слоя (`layers.Dense`) с последовательным уменьшением числа нейронов и функциями активации `relu` или `elu`.
- Выход кодировщика: слой латентного пространства (размерность `latent_dim` должна быть значительно меньше, чем `num_features`).

2. Decoder (Декодировщик):

- Принимает на вход сжатый латентный код.
- Симметрично повторяет архитектуру кодировщика в обратном порядке (постепенно увеличивая размерность).
- Выходной слой: должен строго иметь размерность `num_features`.

Важное правило выбора активации на выходе декодировщика:

*Если входные данные были нормализованы в диапазон $[0, 1]$, используйте функцию активации **sigmoid**.*

*Если данные были стандартизованы (содержат отрицательные значения), оставьте активацию выходного слоя **линейной**.*

Шаг 3. Сборка и обучение модели

1. Объедините спроектированные Энкодер и Декодер в единую сквозную модель `Autoencoder`.
2. Скомпилируйте модель, выбрав в качестве функции потерь среднеквадратичную ошибку `loss='mse'` и оптимизатор `optimizer='adam'`.
3. Запустите процесс обучения.

Методологическая деталь: Так как автокодировщик обучается восстанавливать свои собственные входные данные без использования внешних меток, в метод `model.fit(X, Y)` в качестве целевой переменной (таргета `Y`) необходимо передать саму матрицу признаков `X`.

```
# Пример компиляции и запуска обучения
autoencoder.compile(optimizer='adam', loss='mse')
```

```
history = autoencoder.fit(
    X_train, X_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_val, X_val)
)
```

Шаг 4. Оценка качества и анализ результатов

1. Постройте графики изменения функции потерь (loss и val_loss) по ходу эпох обучения. Проанализируйте сходимость алгоритма.
2. Пропустите тестовую выборку через обученную модель автокодировщика. Вычислите финальную метрику **MSE** между исходными и восстановленными векторами признаков. Сделайте краткий вывод об успешности восстановления данных.